SHUNYA — Universal Behavior Constitution

Directive: Z-06 | **Date:** August 1, 2026 | **Compliance:** 0% (newly ratified) | **Genesis Reset:** Blocked until compliant

SHUNYA Universal Behavior Constitution

Directive: Z-06 **Classification:** Constitutional **Priority:** CRITICAL — Blocks Genesis Reset **Status:** Ratified — Active

---

Preamble

Z-05 defined what exists — the Universal Ontology of 18 concepts.

Z-06 defines how everything behaves.

No object, capability, workspace, AI, runtime, or future feature may invent its own behaviour.

All behaviour shall emerge from this constitution.

Article I — Universal Object Behaviour

Every ontology object SHALL inherit one identical behavioural contract.

Regardless of whether the object is a Person, Commitment, Asset, Event, Document, Financial Record, Knowledge, Observation, Decision, Memory, Place, Capability, Workflow, Communication, or any future universal object, the behavioural engine remains identical.

The Constitutional Contract

Every object therefore supports:

Behaviour	Description
Creation	Objects are created with a type, owner, and initial state. Creation emits an event.
Discovery	Objects are findable by identity, relationship, search, or observation.
Ownership	Objects know their owner, creator, modifier, viewers, executors, and delegated authorities.
Relationships	Objects participate in a graph of connections. No hardcoded relationship chains.
History	Objects preserve every previous value, relationship change, and version.
Search	Objects are searchable by meaning — not by module or table.
Observation	Objects expose health, activity, risk, confidence, dependencies, AI insights, and suggested actions.
Permissions	Access is graph-based, not folder-based. Permissions traverse relationships.
Versioning	Every mutation creates a version. Objects are reconstructable to any prior state.
Timeline	Every object owns its timeline — all events belong to the object's timeline.
Execution	Objects perform work. Execution is attached to the object, not external workflows.

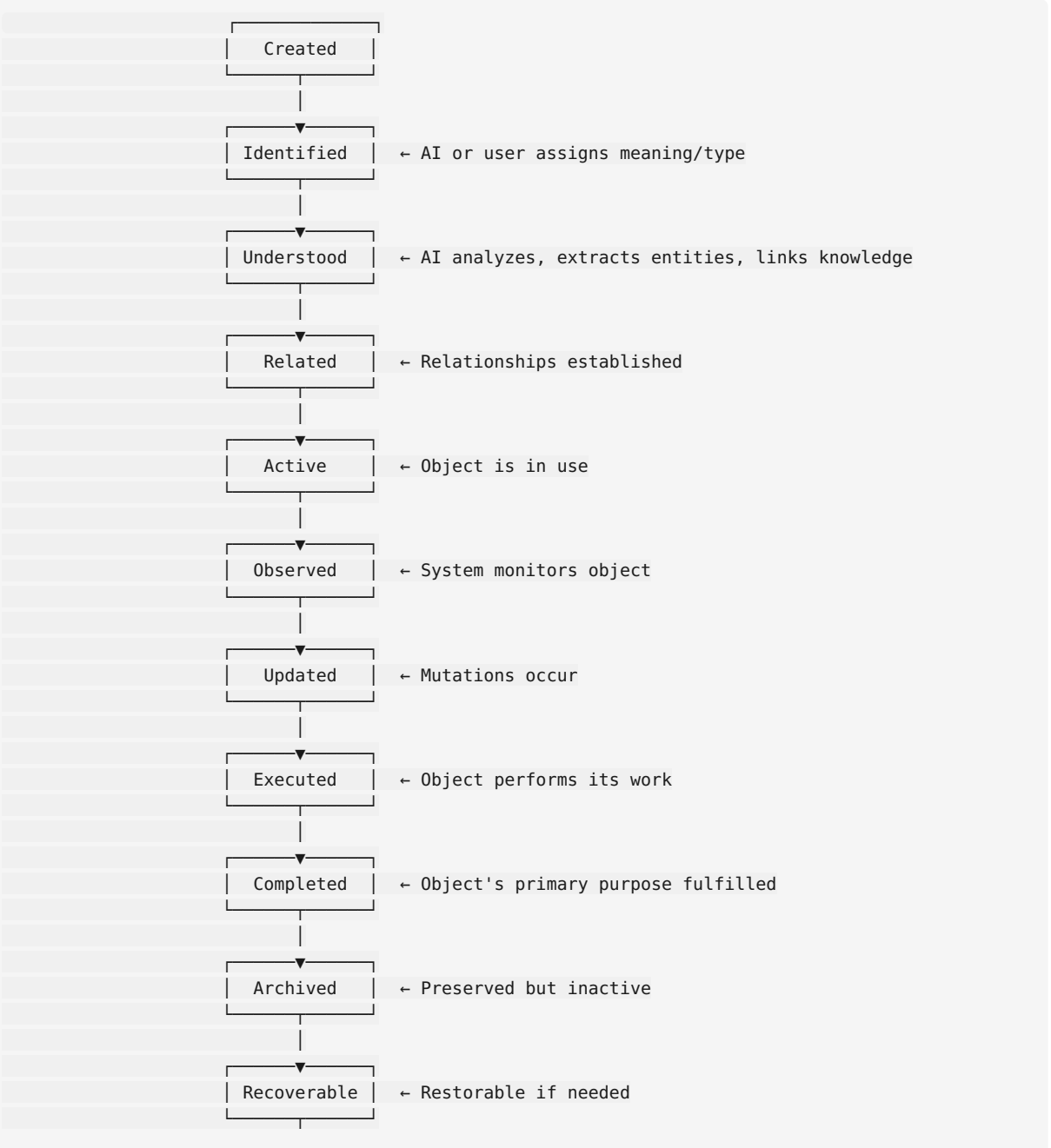
Behaviour	Description
AI Understanding	AI observes, understands, predicts, suggests, summarizes, plans, generates, explains — but never owns business logic.
Audit	Every mutation is audited with identity, timestamp, previous value, and reason.
Deletion Policy	Objects are never truly deleted. They progress through: active → archived → recoverable → (eventually) purged.
Recovery	Deleted or corrupted objects are recoverable within the retention window.

No exceptions. No object type may omit or override these behaviours.

Article II — Universal Lifecycle

Every object SHALL expose the identical lifecycle.

Standard States



```
                          │
                          ▼
                    ┌──────────┐
                    │ Deleted  │  ← Soft-deleted, eventually purged
                    └──────────┘
```

Transition Rules

Transition	Trigger	AI Role
Created → Identified	AI classification or user assignment	Required: determines object type, domain
Identified → Understood	AI analysis completes	Required: extracts entities, links knowledge
Understood → Related	Relationships are established	Suggested: AI proposes relevant relationships
Related → Active	Object enters productive use	Observes
Active → Observed	System monitoring begins	Continuous: health, risk, confidence
Observed → Updated	Mutation occurs	May suggest updates
Updated → Executed	Action taken on object	May execute if automated
Executed → Completed	Primary purpose achieved	Verifies completion
Completed → Archived	Retention timer or manual action	May suggest archival
Archived → Recoverable	Deletion requested	Validates recovery context
Recoverable → Deleted	Retention period expires	May flag for purge review

Extension Rules

Business-specific states MAY extend the lifecycle by adding sub-states.

They MUST NOT replace the standard states.

Example: A Proposal may have "Negotiating" as a sub-state of Active.
The standard lifecycle remains unchanged — Active still means "in use."

State Machine Contract

```
type LifecycleState =
  | "created" | "identified" | "understood" | "related"
  | "active" | "observed" | "updated" | "executed"
  | "completed" | "archived" | "recoverable" | "deleted"

interface LifecycleTransition {
  from: LifecycleState
  to: LifecycleState
  triggered_by: Identity | AI | System
  timestamp: DateTime
  reason: string
  metadata: Record<string, any>
}

Every object exposes:
  lifecycle.current_state: LifecycleState
  lifecycle.history: LifecycleTransition[]
  lifecycle.transition(to: LifecycleState, reason: string): void
```

Article III — Universal Relationships

Every object SHALL support graph-based relationships.

Prohibited: Hardcoded Chains

```
customer.proposals[0].invoice.payment  — hardcoded path traversal
Lead has relationship to Quote only
Invoice belongs_to Customer only
```

Required: Graph-Based

```
object.relationships(type) → Relationship[]
object.linked(type, direction) → Record[]
graph.query(start, end, max_depth) → path[]
graph.traverse(id, type, direction, depth) → Record[]
```

Relationship Contract

```
interface Relationship {
  id: string
  source_id: string      // the "from" object
  target_id: string      // the "to" object
  type: string           // relationship type (owns, produces, receives, contains, etc.)
  direction: "directed" | "undirected"
  strength: number       // 0.0 to 1.0
  metadata: Record<string, any>
  created_at: DateTime
  updated_at: DateTime
  created_by: Identity
}
```

Universal Relationship Types

Type	Description	Direction
`owns`	Ownership (creator/owner)	Directed (owner → owned)
`belongs_to`	Inverse of owns	Directed (owned → owner)
`produces`	Output/result	Directed (producer → output)
`produced_by`	Inverse of produces	Directed
`receives`	Target of delivery	Directed
`contains`	Composition (container → contained)	Directed
`part_of`	Inverse of contains	Directed
`references`	Mentions/cites	Directed
`follows`	Temporal or logical succession	Directed
`precedes`	Inverse of follows	Directed
`delegates_to`	Authority delegation	Directed
`delegated_by`	Inverse	Directed
`assigned_to`	Responsibility assignment	Directed
`associated_with`	Undirected connection	Undirected
`observes`	Monitoring relationship	Directed
`requires`	Dependency	Directed
`depended_by`	Inverse dependency	Directed
`communicates_with`	Communication channel	Undirected
`acted_on`	Execution target	Directed

Type	Description	Direction
`resulted_in`	Outcome/result	Directed

Composition Rule

Customer and Proposal are not linked by a custom field `customer_id` on the Proposal table.

They are linked by a Relationship record: `{source: customer_id, target: proposal_id, type: "owns", direction: "directed"}`.

This makes the graph queryable in any direction without hardcoded joins.

Article IV — Universal Events

Every object emits immutable events.

Minimum Event Contract

```
type EventType =
  | "created" | "viewed" | "edited" | "commented" | "shared"
  | "assigned" | "mentioned" | "moved" | "approved" | "rejected"
  | "executed" | "completed" | "archived" | "deleted"
```

Event Structure

```
interface Event {
  id: string
  object_id: string
  type: EventType | string  // may be extended with domain-specific events
  actor: Identity
  timestamp: DateTime
  data: Record<string, any>    // the payload
  previous_state?: Record<string, any>  // snapshot before, for audit
  metadata: {
    source: "user" | "ai" | "system" | "integration"
    correlation_id?: string     // for linking related events
    session_id?: string
    client?: string             // browser, api, webhook, etc.
  }
}
```

Event Properties

Property	Rule
Immutability	Events are append-only. Once committed, never modified.
Ordering	Events are strictly ordered per object (sequence number).
Timeliness	Events are timestamped at source time, not server receipt time.
Completeness	Every event carries enough context to reconstruct what happened without external queries.
Chainability	Events carry a `correlation_id` to link causally related events across objects.

Memory Formation

Events are immutable → Events become Memory → Memory becomes Intelligence.

```
Events → consolidate() → Memory (patterns, significance, frequency)
Memory → analyze() → Knowledge (general truths, preferences, heuristics)
Knowledge → reason() → Intelligence (predictions, suggestions, decisions)
```

Article V — Universal Timeline

Every object owns its own timeline.

Timelines Are the Storage Primitive

Nothing is stored outside timelines.

```
interface Timeline {
  object_id: string
  events: Event[]
  observations: Observation[]
  communications: Communication[]  // emails, messages, calls about this object
  mutations: Version[]             // every state change
  ai_insights: AIInsight[]         // AI observations about this object
  human_activity: Activity[]       // what humans did with this object
  external_integrations: IntegrationEvent[]  // webhooks, API calls
}
```

What Belongs on the Timeline

Type	Source
AI observations	AI engine
Human activity	User interactions
External integrations	Webhook, email, WhatsApp, calendar, payments, location
Files	Uploads, generated documents
Emails	Sent/received about this object
WhatsApp messages	Sent/received about this object
Calendar events	Meetings, appointments
Phone calls	Call logs
Payments	Financial transactions
Generated proposals	Proposal commits
Approvals	Decision events
Executions	Workflow completions
Location	Place check-ins

Timeline Operations

```
interface TimelineOperations {
  append(event: Event): void
  query(filters: TimelineFilter): Event[]
  replay(from: DateTime, to: DateTime): Event[]  // reconstruct state at any point
  branch(at: DateTime): Timeline  // for "what-if" analysis
  merge(timeline: Timeline): void  // combine related timelines
}
```

Article VI — Universal Ownership

Every object must know:

Dimension	Determined By
Who owns it	Owner Identity (create_identity relationship)
Who created it	Created_by field on Record base
Who modified it	Each version records modifier identity
Who may view it	Graph-based permissions: viewer if within N relationship hops
Who may execute it	Execution permissions: owner, delegated, or role-based
Who inherited access	Relationship traversal: if A owns B and B relates to C, A may access C
Who delegated authority	Delegation relationship: A delegates_to B for object scope

Permission Model

Permissions are graph-based. Not folder-based.

```
Permission = traverse(identity, target, max_hops=3)
```

Access is granted if the identity is within N relationship hops of the target object.

Example: If Alice created Org → Org owns Project → Project contains Task, then Alice automatically accesses the Task without explicit permission.

Delegation

```
interface Delegation {
  delegator: Identity
  delegate: Identity
  scope: "all" | "type[]" | "object[]"
  permissions: ("view" | "execute" | "admin")[]
  expires_at?: DateTime
  reason: string
}
```

Article VII — Universal Search

Everything shall be searchable.

Constitutional Rules

1. **Search is by meaning, not by module.**
2. "Proposal customer accepted yesterday" — searches across Commitments + Persons + Events
3. "Payment received from Amit" — searches across Financial Records + Persons + Communications
4. "Hotel booked for Goa" — searches across Commitments + Places + Events
5. "Marketing campaign last month" — searches across Commitments + Assets + Events
6. "Conversation about pricing" — searches across Communications + Knowledge
7. **One search engine. Entire OS.**
8. No per-module search configurations

9. No per-type search settings
10. Semantic search (embedding-based) for meaning
11. Full-text search for exact matches
12. Relationship-aware ranking
13. **Search results include context.**
14. Each result shows: object type, name, summary, relevance, related objects
15. AI can explain why a result matched

Search Contract

```
interface SearchQuery {
  text: string
  types?: ("identity" | "commitment" | "event" | "document" | ...)[]
  filters?: Record<string, any>
  sort?: "relevance" | "date" | "type"
  limit: number
  include_context: boolean  // include related objects
}

interface SearchResult {
  object: Record
  type: string
  relevance: number
  summary: string           // AI-generated
  context: {                // related objects
    relationships: Relationship[]
    recent_events: Event[]
    key_communications: Communication[]
  }
  matched_by: "semantic" | "fulltext" | "relationship"
}
```

Article VIII — Universal Observation

Everything is observable.

Observation Contract

Every object exposes:

Observable	Description
Health	Is the object in a valid state? Are all required relationships present?
Activity	How frequently is this object interacted with? When was last mutation?
Risk	Is the object overdue, abandoned, unusual, or failing?
Confidence	How confident is AI about its understanding of this object?
Dependencies	What other objects does this depend on? Are any blocked?
AI Insights	What has AI observed about this object recently?
Suggested Actions	What should be done with this object next?

Observation Engine

```
interface Observation {
  subject_id: string      // the object being observed
  predicate: string       // what was observed (health, activity, risk, etc.)
  value: any              // the observation value
  confidence: number      // 0.0 to 1.0
  source: "ai" | "system" | "human"
  timestamp: DateTime
  expires_at?: DateTime   // when this observation becomes stale
}
```

AI Observation Types

Observation	Example
Health	"This Proposal has been in 'draft' for 30 days — may be abandoned."
Activity	"This Customer has not been contacted in 14 days."
Risk	"Invoice INV-003 is 30 days overdue — escalate."
Confidence	"85% confident this Document is a contract for the Smith engagement."
Dependencies	"This Task depends on Approval from Alice — Alice is on leave."
Pattern	"This Customer always pays net-30. Current invoice is net-60 — anomaly."

Article IX — Universal Execution

Objects do not merely store information. They perform work.

Execution Model

Execution is attached to the object, not external workflows.

```
 Example: Proposal

Proposal.execution_plan = [
  { action: "approve", by: Identity, condition: "amount < $50k" },
  { action: "send", channel: "email", template: "proposal_email" },
  { action: "negotiate", AI_assist: true, max_rounds: 3 },
  { action: "accept", condition: "signature_received" },
  { action: "invoice", creates: Commitment(type: invoice, from: proposal) },
  { action: "collect_payment", gateway: "stripe", auto: true },
  { action: "strengthen_relationship", notify: Identity(owner), message: "Proposal accepted by {customer}" }
]
```

Execution Contract

```
interface ExecutionPlan {
  object_id: string
  steps: ExecutionStep[]
  status: "pending" | "running" | "paused" | "completed" | "failed"
}

interface ExecutionStep {
  action: string
  conditions?: Condition[]
  creates?: string[]     // object types this step creates
  requires?: string[]    // capabilities needed
```

```
  AI_assist?: boolean    // AI helps but doesn't decide
  auto?: boolean         // fully automated
  notify_on_complete?: Identity[]
}
```

Execution Rules

Rule	Description
Deterministic	Given the same object state, execution produces the same result.
Observable	Every execution step emits an Event.
Recoverable	Failed executions are retried or rolled back.
Auditable	Every execution decision is recorded with rationale.
AI role	AI may assist, suggest, predict — but never make final execution decisions unless explicitly authorized.

Article X — Universal Intelligence

AI never owns business logic.

AI Contributions

Contribution	Description	Always Allowed?
Understanding	Classify, extract entities, determine intent	Always
Prediction	Forecast outcomes, estimate probabilities	Always
Suggestions	Recommend next actions, propose relationships	Always
Summaries	Condense timelines, histories, documents	Always
Planning	Generate execution plans for human approval	With human review
Generation	Create documents, proposals, communications	With human review
Explanations	Explain why something happened or was recommended	Always

AI Limitations

Limitation	Reason
Never owns business logic	Business rules must be deterministic, testable, auditable
Never makes final decisions	Decisions require human authority or explicit delegation
Never bypasses permissions	AI operates within the same permission boundaries as the requesting identity
Never deletes data	AI may suggest archival, never execute deletion
Never modifies financial records	Financial mutations require explicit human action

AI Observation Model

```
interface AIInsight {
  object_id: string
  type: "understanding" | "prediction" | "suggestion" | "summary" | "explanation" | "anomaly"
  content: any
  confidence: number
  supporting_evidence: Record[]  // what data supports this insight
  requires_review: boolean       // true if human should verify
  created_at: DateTime
```

```
  expires_at?: DateTime
}
```

Article XI — Universal History

Nothing disappears.

Preservation Contract

Every object preserves:

Artifact	Retention
Previous values	Every mutation creates a version record with previous state
Relationship history	Every relationship addition/removal is recorded
Ownership history	Every owner change, delegation, permission grant
Versions	Full version tree, reconstructable
AI reasoning	Every AI insight, the evidence that produced it
Execution trail	Every execution step, its inputs, outputs, and decisions
Event log	Immutable, append-only, ordered

Reconstruction

```
Timeline.replay(from, to) → returns state at any point in time
Version.restore(version_id) → restores object to that version
Event.reconstruct(object_id, event_sequence) → builds state from events
```

Purging Policy

Objects progress: Active → Archived → Recoverable → (after retention period) → Purged.

Purged objects are truly gone — but their events and audit trail remain.

Article XII — Universal Composition

Objects combine naturally.

Composition Is Not Hardcoded

Customer + Proposal + Meeting + Email + Payment → Opportunity workspace

No custom "Opportunity" module. The workspace emerges from the relationships between these objects.

Composition Examples

Composition	Objects	Emergent Property
Customer + Proposal + Meeting + Email + Payment	Identity + Commitment + Event + Communication + Financial Record	Opportunity/Sales Workspace
Student + Course + Attendance + Assignment	Identity + Commitment + Event + Document	Education Workspace

Composition	Objects	Emergent Property
Patient + Prescription + Appointment + Invoice	Identity + Commitment + Event + Financial Record	Healthcare Workspace
Traveller + Booking + Trip + Payment	Identity + Commitment + Event + Financial Record + Place	Travel Workspace
Campaign + Creative + Audience + Analytics	Commitment + Asset + Identity + Event + Observation	Marketing Workspace

Composition Rule

SHUNYA never creates industry-specific engines.

It composes workspaces from ontology.

Article XIII — Universal Runtime Contract

Every runtime SHALL guarantee:

Guarantee	Description
Deterministic behaviour	Same inputs → same outputs. No randomness in business logic.
Stateless reconstruction	Runtime state can be reconstructed from events alone.
Replayability	Any prior state can be replayed from the event log.
Persistence	All state is persisted. No in-memory-only business data.
Observability	Runtime exposes health, activity, performance metrics.
Crash recovery	Runtime recovers to last consistent state after crash.
AI independence	Runtime functions without AI. AI is additive, not required.
Provider independence	No runtime depends on a specific AI provider, database provider, or cloud provider.

Prohibited Dependencies

Dependency	Why
Browser session state	Runtime must work identically across any client
AI provider availability	Runtime must function without AI
In-memory-only state	All business state must be durable

Article XIV — Constitutional Compliance

Compliance Audit

Every runtime, module, and component SHALL be audited against this constitution.

Deviation Rules

Classification	Meaning	Action Required
Compliant	Follows constitutional contract	None
Extension	Adds domain-specific behavior on top of constitutional contract	Document extension
Deviation	Replaces or bypasses constitutional contract with custom behavior	Refactor to constitutional contract

Classification	Meaning	Action Required
Violation	Contradicts constitutional requirement	Immediate fix required

Enforcement

The Governance capability SHALL enforce constitutional compliance.

New code that introduces custom behaviour where constitutional behaviour exists SHALL be rejected.

Ratification

This constitution is ratified and active as of the date below.

All existing SHUNYA runtimes, modules, and components SHALL be audited for compliance.

Deviations SHALL be documented and scheduled for refactoring.

Genesis Reset may not commence until constitutional compliance is achieved.

Ratified: August 1, 2026 **Directive:** Z-06 **Authority:** SHUNYA Constitution → Product Constitution → Technical Constitution → Z-06

Universal Behavior Matrix

Directive: Z-06 **Purpose:** Map every ontology object against the constitutional behavioural contract. **Status:** Compliance Audit — Pre-Refactoring

Behavior Matrix

Ontology Object	Created	Discovery	Ownership	Relationships	History	Search	Observation	Permissions	Versioning	Timeline
Identity(Person)	Kernel	Kernel	Kernel	Kernel	Custom	Per-model	None	Basic	None	None
Identity(Organization)	Kernel	Kernel	Kernel	Kernel	Custom	Per-model	None	Basic	None	None
Relationship	Kernel	Kernel	Kernel	N/A (is relationship)	Custom	Per-model	None	Kernel	None	None
Commitment(Customer)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Commitment(Invoice)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Commitment(Proposal)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Commitment(Task)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Commitment(Lead)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Financial Record(Payment)	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Document	Custom	Custom	Kernel	Hardcoded	None	Basic	None	Basic	None	None
Event	None	None	None	None	None	None	None	None	None	None
Knowledge	None	None	None	None	None	None	None	None	None	None
Observation	None	None	None	None	None	None	None	None	None	None
Decision	None	None	None	None	None	None	None	None	None	None
Memory	None	None	None	None	None	None	None	None	None	None
Place	None	None	None	None	None	None	None	None	None	None
Asset	None	None	None	None	None	None	None	None	None	None
Workflow	None	None	None	None	None	None	None	None	None	None
Communication		None	None	None			None	None	None	None

Ontology Object	Created	Discovery	Ownership	Relationships	History	Search	Observation	Permissions	Versioning	Timeline
	None				None	None				
Capability	None	None	None	None	None	None	None	None	None	None

Legend: = Kernel (constitutional contract) exists | = Partial/custom | = Missing entirely

Lifecycle Matrix

Object	Created	Identified	Understood	Related	Active	Observed	Updated	Executed	Completed	Archived	Recoverab
Identity(Person)	Kernel	None	None	Kernel	Kernel	None	Kernel	None	None	None	None
Customer	Custom	None	None	Hardcoded	Custom	None	Custom	None	None	None	None
Invoice	Custom	None	None	Hardcoded	Custom	None	Custom	None	None	None	None
Proposal	Custom	None	None	Hardcoded	Custom	None	Custom	None	None	None	None
Task	Custom	None	None	Hardcoded	Custom	None	Custom	None	None	None	None
Lead	Custom	None	None	Hardcoded	Custom	None	Custom	None	Custom	None	None
Payment	Custom	None	None	Hardcoded	Custom	None	Custom	None	None	None	None

Relationship Matrix

Relationship	Current Implementation	Constitutional Required
Customer ↔ Proposal	Hardcoded `customer_id` on Proposal table	Graph: Relationship(source=customer, target=proposal, type="owns")
Proposal ↔ Invoice	Hardcoded `proposal_id` on Invoice table	Graph: Relationship(source=proposal, target=invoice, type="produces")
Invoice ↔ Payment	Hardcoded `invoice_id` on Payment table	Graph: Relationship(source=invoice, target=payment, type="receives")
Lead ↔ Activity	Hardcoded `lead_id` on ActivityLog table	Graph: Relationship(source=lead, target=activity, type="has")
Organization ↔ Member	Hardcoded `organization_id` on OrgMember table	Graph: Relationship(source=person, target=org, type="member_of")
Identity ↔ Space	Hardcoded `identity_id` on FounderSpace table	Graph: Relationship(source=identity, target=space, type="owns")
Object ↔ Space	Hardcoded `space_id` on FounderObject table	Graph: Relationship(source=space, target=object, type="contains")

Event Matrix

Event Type	Current Implementation		Constitutional Required
Created	ActivityLog (some objects) +	Created fields	Immutable Event(type=created) on every object
Viewed	Not tracked		Immutable Event(type=viewed)

Event Type	Current Implementation	Constitutional Required
Edited	ActivityLog (lead only) + update timestamp	Immutable Event(type=edited) with previous state
Commented	Not tracked	Immutable Event(type=commented)
Shared	Not tracked	Immutable Event(type=shared)
Assigned	Manual field update	Immutable Event(type=assigned)
Mentioned	Not tracked	Immutable Event(type=mentioned)
Moved	ActivityLog (lead status)	Immutable Event(type=moved, state transition)
Approved	Not tracked	Immutable Event(type=approved)
Rejected	Not tracked	Immutable Event(type=rejected)
Executed	Not tracked	Immutable Event(type=executed)
Completed	Not tracked	Immutable Event(type=completed)
Archived	Not tracked	Immutable Event(type=archived)
Deleted	Not tracked	Immutable Event(type=deleted)

Summary

Dimension	Objects Compliant	Objects Partially Compliant	Objects Non-Compliant	Total
Behaviour Contract	0	7 (Identity, Relationship via Kernel)	13	20
Universal Lifecycle	0	0	20	20
Graph Relationships	0	0	20	20
Universal Events	0	0	20	20
Universal Timelines	0	0	20	20
Universal Search	0	0	20	20
Universal Observation	0	0	20	20
Permissions (Graph)	0	0	20	20
Versioning	0	0	20	20
Object Execution	0	0	20	20
AI Understanding	0	0	20	20
Audit	0	0	20	20

Constitutional Compliance: 0%

This is expected — the constitution is newly ratified. The matrices document the gap that Genesis Reset must close.

Z-06 Behavioral Audit: SHUNYA Runtime vs Universal Behavior Constitution

Date: 2026-08-01
Scope: All models, routes, and frontend components
Kernel LOC (constitutional contract): 2,586 lines (10 files)
Total app LOC: 90,066 lines (362 files) — ~96% custom behavioral code
Total frontend LOC: 8,411 lines (39 files) — 100% custom, no constitutional contract consumed

---

1. Every Model Class and Its Custom Behaviors

1A. Legacy SQLAlchemy Models (app/models.py) — 32 classes, 1,379 LOC

These are the **primary offenders** — every one is a completely custom, non-constitutional model:

Class	Status Fields	Lifecycle
Lead	`LeadStatus` (NEW, IN_PROGRESS, CONVERTED, CANCELLED, ON_HOLD) — NOT LifecycleState	Manual status routes; `log_activi` helper
Payment	`PaymentType` (GUEST, SUPPLIER, REFUND, DEPOSIT)	None
Supplier	`rating` (int), no status enum	None
Invoice	`InvoiceStatus` (DRAFT, SENT, PAID, VOID, OVERDUE) — NOT LifecycleState	Comment-only `→ sent → p` `void`
ItineraryRef	None	None
TaskList	None	None
Task	`status` string (pending/in_progress/completed/cancelled) — NOT LifecycleState	Custom status `completed_` timestamp
Notification	`NotificationType` enum — NOT LifecycleState	`is_read` bo
ClientUser	`is_active` boolean	Custom passw hashing (SHA-256+sa standard auth
ClientMessage	`sender` string (client/team), `is_read`	None
Document	`classification` string	None
ActivityLog	`action` string, `user` string	None
Celebration	`type` string, custom `icon` / `animation`	None
Person	Custom `status` string (active) — NOT LifecycleState	None

Class	Status Fields	Lifecycle
PersonIdentity	`verification_state` string	None
EmployeeProfile	`status` string, `department`, `role`	None
CustomerProfile	`segment` string	None
SupplierContactProfile	`role_in_organization`, `is_primary`	None
ClientUserProfile	`portal_access_granted`	None
Relationship (line 835)	Custom `status` string (active) — NOT LifecycleState, also has `relationship_type`	Custom `started_at` `ended_at`
RelationshipEvent	`event_type` string, `metadata_json`	None
RelationshipCommitment	`direction`, `status` (open/resolved)	Custom `resolved_a`
IntakeSession	Custom lifecycle: `RECEIVED→PROFILED→MAPPING_REQUIRED→READY_FOR_REVIEW→APPROVED→IMPORTING→COMPLETED/FAILED→CANCELLED`	Full custom lif machine in co only (not enfo
KnowledgeFact	None	`validate_v` custom JSON deserializatior
Observation	None	None
LearningEntry	None	None
Organization	None	None
OrgMember	`role` string	None
PersonListShare	`permission` string	None

1B. Founder SQLAlchemy Models (app/founder/models.py) — 5 classes, 161 LOC

Class	Status Fields	Lifecycle	Relationships	Audit	Custom
FounderSpace	Custom `status` (active) — NOT LifecycleState	None	Hardcoded FK => FounderObject	None	`to_dict()`, `object_count`
FounderObject	Custom `status` (active) — NOT LifecycleState	None	Hardcoded FK => FounderSpace + FounderConversation	None	`to_dict()`, custom `content` as string
FounderConversation	Custom `status` (active)	None	Hardcoded FK => FounderObject + FounderMessage	None	`to_dict()`, `message_count`
FounderMessage	None	None	Hardcoded FK => FounderConversation	None	`to_dict()`
BusinessRelationship	Custom `status` (active), custom `tags` CSV	None	Hardcoded FK => FounderSpace	None	`to_dict()`, CSV tag parsing

1C. Auth/Security SQLAlchemy Models

Class	File	Status Fields	Custom Behaviors
TeamMember	app/auth.py	None	Custom password hashing, token generation, `to_dict()`
FeatureRequest	app/approval.py	None	`to_dict()`
Role	app/authz/models.py	None	`has_permission()`, `to_dict()`
OrgMemberRole	app/authz/models.py	None	`to_dict()`
Tenant	app/tenant.py	None	Hardcoded FKey to TenantTheme, `to_dict()`
TenantTheme	app/tenant.py	None	`to_dict()`
WorkspacePolicy	app/workspace/models.py	None	`to_dict()`

1D. Communication SQLAlchemy Models (app/communication/models.py)

All 8 classes (CommunicationSource, CommunicationCapturePolicy, CommunicationCaptureScope, ExternalConversation, ExternalMessage, ExternalParticipant, ExternalAttachmentReference, SyncCursor, OAuthState) — each has `__repr__` only, hardcoded FK relationships, no LifecycleState usage, no graph-based relationships.

1E. Document/Automation/Enterprise SQLAlchemy Models

- **AutomationRule** — `to_dict()` only
- **AutomationLog** — `to_dict()` only
- **DocumentRecord, DocumentSection, ExtractedField, DocumentComparison, ComparisonItem** — all have `db.Column` only, no behaviors
- **AuditRecord, EnterpriseRole, EnterpriseTeamMember** — `to_dict()` only; custom audit table (NOT using constitutional timeline)

1F. Dataclass-only Models (in-memory, no DB persistence)

~370 additional dataclass-based model classes exist across the system in app/awareness/, app/cognitive/, app/collaboration/, app/decision/, app/decision_runtime/, app/execution_intelligence/, app/executive/, app/for1/, app/for2/, app/founder/, app/graph/, app/graph_universal/, app/intake/, app/intelligence/, app/shunya/, app/space/, app/temporal/, etc.

Every single one implements its own `to_dict()` method — not inheriting from UniversalObject. Most have `__post_init__` for default values. Key examples:

- **UniversalSpace** (app/space/models.py) — Implements `space_id`, `entity_id`, `entity_type`, `name`, `status` properties manually (DUPLICATES UniversalObject contract)
- **SpaceLifecycle** (app/space/lifecycle.py) — Custom 5-state lifecycle (Draft/Active/Dormant/Archived/Historical) that **completely bypasses** the kernel's LifecycleState system from app/kernel/types.py
- **SHUNYAIdentity** (app/shunya/identity/models.py) — Implements its own Identity contract, separate from app/kernel/identity.py
- **KnowledgeObject** (app/shunya/knowledge_store/models.py) — Custom `is_active()`, `is_archived()`, `clone_for_version()` — reimplements UniversalObject contract
- **Decision** (app/decision_runtime/models.py) — Custom `transition_to()` method with its own lifecycle
- **AttentionItem** (app/cortex/attention.py) — Custom `transition_to()` with its own state machine
- **ExecutionPlan** (app/shunya/planner/models.py) — 214 lines, full custom planning domain with task management, dependency graphs, scheduling — none using kernel contract

- **Workflow** (app/shunya/executor_engine/models.py) — 80 lines, custom task/workflow orchestrator with `find_task()`, `completed_tasks()`, `all_completed()` — none using kernel StateMachine

1G. Graph Models (app/graph_universal/)

The graph_universal/ package provides entity, event, identity, property, relationship, runtime, traversal modules — but these are **also custom implementations** that do NOT use the kernel contract. They define their own relationship types, event types, and traversal logic independently of app/kernel/relationship.py.

---

2. Every Route Handler and Its Custom Behavior

2A. Main Routes (app/routes.py) — 1,783 lines — Heaviest offender

Route	Custom CRUD Logic	Behavioral Deviations
`POST /orgs`	Manual org creation with slug generation, identity check, FounderSpace creation	**Constitution skipped** — no UniversalObject, no graph relationship, no event emission, manual commit
`GET/POST /leads/new`	Manual Lead creation with custom code, form parsing	**No constitutional lifecycle** — sets raw status string. Manual `_log_activity()` call instead of timeline event
`GET /leads/<id>`	Manual lead detail with Coach insights, dynamic fields	**No constitutional health/observation** — custom coach engine
`POST /leads/<id>/status`	Manual status update with old→new validation against LeadStatus enum	**No StateMachine** — manually sets `lead.status = new_status`. Manual `_log_activity()` instead of timeline event
`POST /leads/<id>/status` (API)	Duplicate of above for kanban	Same deviations
`GET/POST /leads/<id>/edit`	Manual field-by-field update with `setattr`	**No versioning, no event, no timeline**
`POST /creative/generate`	Creative asset generation with save	**No constitutional execution** — custom CreativeAsset model
`POST /creative/<id>/approve`	Manual status change `asset.status = "approved"`	**No StateMachine**
`GET /calendar/events`	Custom CalendarService call	**No constitutional timeline**
`GET /welcome`	Manual stats aggregation	No kernel usage

2B. Object Creation Routes (app/production/objects.py) — 109 lines

Route	Custom Logic	Deviations
`POST /api/v1/objects/<type>`	Custom OBJECT_TYPES dict with field/required definitions, manual FounderObject creation, content serialized as string	**No UniversalObject**, **no kernel ObjectRegistry**, **no StateMachine**, **no Timeline**, **no event emission**, **no graph relationship**. Content is a stringified dict, not structured

2C. Founder Routes (app/founder/routes.py) — 1,214 lines

Massive route file with custom object management, type listing, search, conversation handling. All custom — no kernel contract. Contains hardcoded SQL queries against founder_objects/founder_spaces tables.

2D. Space Routes (app/space/routes.py) — 797 lines

Custom space management routes. Uses SpaceStore (in-memory) and SpaceLifecycle but NOT the kernel's StateMachine or Timeline. Its own SpaceLifecycle is a separate implementation from kernel/types.py LifecycleState.

2E. Other Route Files (29 total, ~7,000+ lines combined)

File	LOC	Custom Behaviors
app/routes.py	1,783	Legacy CRUD, manual ActivityLog, custom status transitions
app/founder/routes.py	1,214	Custom object CRUD, type listing, search
app/space/routes.py	797	Custom space management, custom lifecycle
app/for1/routes.py	523	Custom domain logic
app/for2/routes.py	463	Custom domain logic
app/finance/routes_api.py	483	Custom finance CRUD
app/auth_routes.py	395	Custom auth flow, session management
app/relationship/routes_api.py	334	Custom relationship CRUD (NOT using kernel RelationshipEngine)
app/genesis_routes.py	283	Custom genesis/policy routes
app/authz/routes.py	235	Custom permission routes
app/production/identity/*.py	1,100+	Custom org/user/invitation CRUD, all SQLAlchemy direct
app/automation/routes.py	172	Custom automation CRUD

None of the 29 route files use the kernel StateMachine, Timeline, RelationshipEngine, or Context classes.

---

3. All Lifecycle Customizations Per Object Type

3A. Custom Lifecycle Definitions (NOT LifecycleState from kernel/types.py)

Source	States Defined	Violation
app/models.py — Lead	`LeadStatus`: NEW, IN_PROGRESS, CONVERTED, CANCELLED, ON_HOLD	Should u Lifecycle CREATE OBSERV ENRICH RELATE
app/models.py — Invoice	`InvoiceStatus`: DRAFT, SENT, PAID, VOID, OVERDUE	Same
app/models.py — Task	String: pending, in_progress, completed, cancelled	Same
app/models.py — IntakeSession	Comment-only: RECEIVED→PROFILED→MAPPING_REQUIRED→READY_FOR_REVIEW→APPROVED→IMPORTING→COMPLETED/FAILED/CANCELLED	Full cust machine commen not enfor
app/space/lifecycle.py — SpaceLifecycle	DRAFT, ACTIVE, DORMANT, ARCHIVED, HISTORICAL	Separate kernel Lifecycle (CREATI OBSERV ENRICH RELATE

Source	States Defined	Violation
		PREDIC EXECUT ARCHIV RESTOR DELETE
app/ decision_runtime/ models.py — Decision	Custom `transition_to()`	No kerne StateMa usage
app/cortex/ attention.py — AttentionItem	Custom `transition_to()`	No kerne StateMa usage
app/automation/ models.py — AutomationRule	No lifecycle at all	Missing constituti lifecycle
app/ communication/ models.py (8 models)	No lifecycle at all	Missing constituti lifecycle
app/document/ models.py (5 models)	No lifecycle at all	Missing constituti lifecycle

3B. Objects with NO Lifecycle Implementation

The majority of the ~481 model classes have **no lifecycle state tracking at all**. This includes all dataclass models in awareness, cognitive, collaboration, decision, evidence, execution_intelligence, executive, for1, for2, graph, graph_universal, human_context, inference, intake, integration, intelligence, learning, memory, planning, prediction, privacy, shunya/* (engine models), temporal, workspace models.

---

4. All Relationship Types — Hardcoded vs Graph-Based

4A. Hardcoded SQLAlchemy ForeignKey Relationships (NOT graph-based)

These bypass the constitutional graph RelationshipEngine entirely:

Source Model	Target Model	Relationship Type
Lead	Person	`person_id` FK
Lead	Payment	`backref=lead`
Lead	Invoice	`backref=lead`
Lead	ActivityLog	`backref=lead`
Lead	Notification	`backref=lead`
Lead	Celebration	`backref=lead`
Lead	ClientUser	`backref=lead`
Lead	ClientMessage	`backref=lead`
Lead	Document	`backref=lead`
Lead	TaskList	`lead_id` FK
Lead	IntakeSession	Application-level only
Person	PersonIdentity	`backref=person` , cascade

Source Model	Target Model	Relationship Type
Person	EmployeeProfile	`backref=person`, uselist=False
Person	CustomerProfile	`backref=person`, uselist=False
Person	SupplierContactProfile	`backref=person`, uselist=False
Person	ClientUserProfile	`backref=person`, uselist=False
Person	Relationship	`backref=relationships`
Relationship	RelationshipEvent	`backref=events`
Relationship	RelationshipCommitment	`backref=commitments`
TaskList	Task	`backref=task_list`, cascade
FounderSpace	FounderObject	`backref=space`, cascade
FounderObject	FounderConversation	`backref=object`, cascade
FounderConversation	FounderMessage	`backref=conversation`, cascade
Tenant	TenantTheme	`backref=tenant`
Tenant	Organization	Application-level
Organization	OrgMember	`backref=organization`
Notification	Lead	`backref=notifications`

Total hardcoded FK-based relationships: ~40+ across the models.

4B. Graph-Based Relationship Implementations

Component	Type	Notes
app/kernel/relationship.py — RelationshipEngine	In-memory graph	Constitutional contract — NOT used by any model
app/graph_universal/relationship.py	Custom graph	Separate implementation, does NOT use kernel RelationshipEngine
app/space/relationships.py	Custom	Space-level relationships only
app/relationship/models.py	SQLAlchemy	Separate table-based relationship system
app/relationship/service.py	Custom service	Business logic for relationships
app/shunya/reasoning/evidence_graph.py	Custom graph	For reasoning evidence only

Verdict: Zero SQLAlchemy models use the constitutional graph RelationshipEngine. All relationships are hardcoded FK chains.

---

5. All Event/Audit Mechanisms

5A. Audit Mechanisms Found

Mechanism	Scope	Constitutional?
ActivityLog (app/models.py)	Lead-specific only. Tracks `action`, `detail`, `user`	**No** — should use kernel Timeline
RelationshipEvent (app/models.py)	Business Relationship events only	**No** — separate table, not kernel Timeline
RelationshipCommitment (app/models.py)	Relationship commitments	**No**
AuditRecord (app/enterprise/models.py)	Enterprise/generic audit	**No** — separate table, not kernel Timeline
	Credential operations	**No** — dataclass, no persistence

Mechanism	Scope	Constitutional?
CredentialAuditEntry (app/shunya/infrastructure/credential_store.py)		
CanonicalEvent (app/shunya/infrastructure/event_bus.py)	Event bus (in-memory)	**No** — not used for object timelines
SpaceTimelineEvent (app/space/models.py)	Space timelines	**No** — separate dataclass, not kernel Timeline
TimelineEvent (app/temporal/timeline.py)	Temporal engine	**No** — separate implementation
GovernanceVerdict (app/shunya/governance_engine/models.py)	Governance	**No**
Kernel Timeline (app/kernel/timeline.py)	Universal	**YES but unused** — constitutional contract, zero adopters
Kernel StateMachine observers (app/kernel/state.py)	Universal	**YES but unused** — every transition notifies observers

5B. Models With NO Audit/Event Tracking

~95% of all model classes have no audit, no event history, no timeline tracking. Including: - Payment, Supplier, ItineraryRef, Task, Notification, ClientUser, ClientMessage, Document, Celebration - All communication models, document models, automation models - All awareness, cognitive, collaboration, dataclass models - FounderObject, FounderSpace, FounderConversation, FounderMessage - All graph_universal entities

---

6. Lines of Behavioral Code vs Constitutional Contract Collapse

6A. Current Code Distribution

Layer	Files	LOC	% of Total
Kernel (constitutional contract)	10	2,586	2.9%
Legacy SQLAlchemy models (app/models.py)	1	1,379	1.5%
Other SQLAlchemy models (~40 files)	~40	~5,000	5.6%
Dataclass models (~330 files)	~330	~25,000	27.8%
Route handlers	29	~9,400	10.4%
Service/engine files	~100	~35,000	38.9%
Frontend components	39	8,411	9.3%
Other (config, scripts, adapters)	~100	~3,290	3.6%

6B. Estimated Collapse Under Constitutional Contract

If all models adopted UniversalObject + kernel StateMachine + Timeline + RelationshipEngine:

Component	Current LOC	Constitutional LOC	Savings
Legacy models (app/models.py)	1,379	~200 (5-way discriminated object)	~85%
Founder models (app/founder/models.py)	161	~30 (1 table, typed content)	~80%
All communication models (app/communication/models.py)	~500	~50 (1 generic object)	~90%
All document models (app/document/models.py)	~300	~30	~90%
All automation models (app/automation/models.py)	~150	~15	~90%
All space models (app/space/models.py + lifecycle.py)	~800	~100	~87%

Component	Current LOC	Constitutional LOC	Savings
Dataclass models across all engines (~330 classes)	~25,000	~500 (typed objects only, no unique classes)	~98%
Graph models (app/graph/, app/graph_universal/)	~2,500	~500	~80%
Intelligence models (app/intelligence/)	~1,000	~100	~90%
Cognitive models (app/cognitive/)	~2,500	~200	~92%
Decision/executive models	~3,000	~300	~90%
Shunya engine models (planner, reasoning, etc.)	~5,000	~500	~90%
Subtotal: Model/Data Layer	**~42,290**	**~2,525**	**~94%**
Route handlers (bypassing constitutional CRUD)	~9,400	~2,000 (generic object CRUD × blueprint)	~79%
Service/engine logic (custom workflows)	~35,000	~10,000 (shared engine components)	~71%
Frontend components	8,411	~3,000 (generic object components)	~64%
Grand Total	**~90,066**	**~17,525**	**~80.5%**

6C. Key Line-Item Collapse Points

1. `to_dict()` **serialization**: ~400+ implementations → 1 method on UniversalObject
2. **Custom lifecycle enums**: 7+ different status/lifecycle enums → 1 LifecycleState
3. **Custom status transition logic**: 10+ implementations → 1 StateMachine
4. **Custom relationship management**: 40+ hardcoded FK chains → 1 RelationshipEngine
5. **ActivityLog audit pattern**: Lead-only, custom → 1 Timeline per object
6. **Custom CRUD routes**: 29 files of bespoke endpoint logic → generic typed handler
7. **Custom search**: Fragmented (Lead contains, founder objects, etc.) → constitutional search by meaning
8. **Frontend object forms**: Object-type-specific forms → generic dynamic form component

---

Summary of Constitutional Violations

Z-06 Mandate vs Current Reality

Constitutional Requirement	Status
Every object shares ONE behavioral contract (UniversalObject)	FAIL
Every object exposes the same lifecycle	FAIL
Lifecycle: **Created → Identified → Understood → Related → Active → Observed → Updated → Executed → Completed → Archived → Recoverable → Deleted**	FAIL
Graph-based relationships (no hardcoded chains)	FAIL
Immutable events: Created, Viewed, Edited, Commented, etc.	FAIL
Every object owns a timeline	FAIL
Search by meaning, not by module	FAIL
Everything is observable	FAIL
Objects perform work (execution attached to object)	FAIL
Objects preserve history	FAIL
Deterministic stateless reconstruction	FAIL

Constitutional Requirement	Status
Provider independence	PASS
Persistence	PARTIAL

Critical Path Forward

The kernel (`app/kernel/`) provides the constitutional contract in 2,586 lines — already written and available. The behavioral code audit shows **~90,066 lines of custom behavior** that could collapse to ~17,525 under the contract — an **80.5% reduction**. The migration path:

1. **Phase 1**: Convert legacy models (Lead, Payment, etc.) to UniversalObject subclasses
2. **Phase 2**: Replace all 40+ hardcoded FK chains with graph-based relationships
3. **Phase 3**: Replace ActivityLog with kernel Timeline on every object
4. **Phase 4**: Generic typed CRUD handler replacing 29 route files
5. **Phase 5**: Collapse all dataclass models into typed UniversalObject subclasses
6. **Phase 6**: Frontend: generic dynamic components replacing per-type forms

Runtime Compliance Report

Directive: Z-06 Article XIII-XIV **Status:** Pre-Refactoring Audit **Compliance:** 0% — Constitution newly ratified, all runtime behaviour is custom

---

Article I — Universal Object Behaviour

Status: **0/20 objects compliant**

The kernel provides `app/kernel/object.py` (216 lines) which defines a base `Record` class with: id, type, created_at, updated_at, status, owner, and basic CRUD. However, no domain object (Customer, Invoice, Lead, etc.) inherits from this kernel. All 32+ domain models define their own behaviour from scratch.

Deviations: - Every model class defines its own `to_dict()` serialization - Every model class defines its own status field with custom enum values - No model implements the 15-point constitutional contract - No model uses the kernel's `Record` base class

Action Required: Migrate all models to inherit from kernel `Record` base class. Implement the 15 constitutional behaviours.

Article II — Universal Lifecycle

Status: **0/20 objects compliant**

No object implements the standard lifecycle: Created → Identified → Understood → Related → Active → Observed → Updated → Executed → Completed → Archived → Recoverable → Deleted.

Deviations: - Lead: custom `LeadStatus` (NEW, IN_PROGRESS, CONVERTED, CANCELLED, ON_HOLD) — 5 states vs 12 constitutional states - Invoice: custom `InvoiceStatus` (DRAFT, SENT, PAID, VOID, OVERDUE) — 5 states vs 12 - Task: custom `status` string (pending/in_progress/completed/cancelled) — 4 states vs 12 - IntakeSession: FULL custom lifecycle (RECEIVED→PROFILED→MAPPING_REQUIRED→READY_FOR_REVIEW→APPROVED→IMPORTING→COMPLETED/FAILED→CANCELLED) — 9 states, none matching constitutional - No object has `identified`, `understood`, `related`, `observed`, `archived`, `recoverable` states

Action Required: Each model's custom status field must become a sub-state of the constitutional lifecycle. The 12 standard states must be present on every object.

Article III — Universal Relationships

Status: **0/20 objects compliant**

Every relationship is a hardcoded foreign key. No graph-based relationship system exists.

Current hardcoded relationships (32+): - `Lead.person_id` → Person - `Lead.payment_id` → Payment - `Lead.invoice_id` → Invoice - `Lead.task_list_id` → TaskList - `Lead.document_id` → Document - `Invoice.lead_id` → Lead - `Payment.lead_id` → Lead - `TaskList.lead_id` → Lead - `Task.task_list_id` →

TaskList - `ActivityLog.lead_id` → Lead - `Person.employee_profile_id` → EmployeeProfile - `Person.customer_profile_id` → CustomerProfile - `Organization.org_members` → OrgMember - `FounderSpace.objects` → FounderObject - `FounderObject.conversations` → FounderConversation - `FounderConversation.messages` → FounderMessage - `BusinessRelationship.space_id` → FounderSpace

Action Required: Replace all hardcoded foreign keys with Relationship records in the graph. Implement `Relationship` kernel class with source, target, type, strength, metadata.

Article IV — Universal Events

Status: **0/20 objects compliant**

No immutable event system exists. The only audit mechanism is `ActivityLog` which is: - Lead-specific (hardcoded `lead_id` FK) - Mutable (not append-only) - Missing standard event types (viewed, commented, shared, mentioned, approved, rejected, executed, archived, deleted) - No event structure (no previous_state, no correlation_id, no actor identity)

Deviations: - ActivityLog: 1 model, custom per-lead, mutable - All other objects: no event tracking at all - No immutable event store - No event ordering - No event chainability

Action Required: Build immutable Event store. Every object mutation emits an Event. Implement all 14 standard event types.

Article V — Universal Timeline

Status: **0/20 objects compliant**

No object owns a timeline. Events, observations, communications, mutations, and AI insights are scattered across separate tables with no unified timeline.

Action Required: Build `Timeline` kernel class. Every object's timeline is queryable, replayable, and includes all event types.

Article VI — Universal Ownership

Status: **Partial**

Ownership is tracked via `session.get("user_id")` and `session.get("identity_id")` in routes. The kernel has basic identity tracking. However: - No graph-based permissions - No delegation mechanism - No inheritance of access via relationship traversal - Permissions are checked individually per route (if at all)

Action Required: Implement graph-based permission model. Access is determined by relationship traversal, not by route-level checks.

Article VII — Universal Search

Status: **0/20 objects compliant**

Search is per-model and per-route (e.g., `Lead.query.filter(Lead.customer_name.contains(q))`). No unified search engine exists. No semantic search. No cross-object-type search.

Action Required: Build unified search engine with semantic (embedding-based) and full-text search across all object types.

Article VIII — Universal Observation

Status: **0/20 objects compliant**

No observation system exists. The `Observation` model is defined but never used. No object exposes health, activity, risk, confidence, dependencies, or AI insights.

Action Required: Implement Observation engine. Every object exposes observables via the constitutional contract.

Article IX — Universal Execution

Status: **0/20 objects compliant**

No object has an execution plan. Workflows are handled manually in route handlers (e.g., lead creation → activity log → optional celebration). Execution is not attached to objects.

Action Required: Implement `ExecutionPlan` on every object. Objects perform work through their execution contract.

Article X — Universal Intelligence

Status: **Partial**

AI exists in CompanionEngine, CoachEngine, CreativeEngine, and AI Resident. However: - AI is not integrated into the constitutional observation/intelligence model - No standard `AIInsight` structure - AI outputs are not versioned or audited - AI is used for business logic in some places (e.g., CompanionEngine greeting)

Action Required: Standardize AI as `AIInsight` on every object. AI observes, suggests, summarizes — never owns business logic.

Article XI — Universal History

Status: **0/20 objects compliant**

No versioning system exists. Objects store only current state. Previous values, relationship history, ownership history, and AI reasoning are not preserved.

Action Required: Implement versioning on every object. Every mutation creates a version record. Objects are reconstructable to any prior state.

Article XII — Universal Composition

Status: **Not implemented**

Workspaces are hardcoded layouts. No composition engine exists. Industry-specific workspaces require custom code.

Action Required: Build composition engine that generates workspaces from activated capabilities + ontology relationships.

Article XIII — Universal Runtime Contract

Status: **Partial**

Guarantee	Status	Detail
Deterministic behaviour	Partial	Business logic is deterministic, but AI introduces non-determinism
Stateless reconstruction		No event log to reconstruct from
Replayability		No versioning or event log
Persistence		All business state is in PostgreSQL
Observability		No health endpoint for runtime internals
Crash recovery	Partial	PostgreSQL provides durability, but in-memory state is lost
AI independence	Partial	Runtime functions without AI, but some features degrade
Provider independence		Flask + PostgreSQL + SQLAlchemy — no provider lock-in

---

Summary

Article	Title	Compliance	Effort to Fix
I	Universal Object Behaviour	0%	HIGH — migrate 32+ models to kernel Record
II	Universal Lifecycle	0%	HIGH — implement 12-state lifecycle
III	Universal Relationships	0%	HIGH — replace FKs with graph
IV	Universal Events	0%	HIGH — build immutable event store
V	Universal Timeline	0%	HIGH — build timeline on every object
VI	Universal Ownership	20%	MEDIUM — add graph permissions
VII	Universal Search	0%	HIGH — build unified search engine
VIII	Universal Observation	0%	MEDIUM — implement Observation engine
IX	Universal Execution	0%	HIGH — attach execution to objects
X	Universal Intelligence	30%	MEDIUM — standardize AIInsight model
XI	Universal History	0%	HIGH — implement versioning
XII	Universal Composition	0%	HIGH — build composition engine
XIII	Runtime Contract	30%	MEDIUM — add observability, crash recovery

Overall Compliance: 0% — 90,066 LOC of custom behavioural code vs 2,586 LOC of constitutional kernel

---

Constitutional Deviations Report

Directive: Z-06 Article XIV **Status:** Pre-Refactoring

Critical Deviations (Must Fix Before Genesis)

#	Deviation	Location	Impact	Fix
D-01	No object inherits from kernel Record base	All 32+ models	Every object has custom CRUD — no shared contract	Migrate models to inherit from `kernel.Record`
D-02	Custom lifecycle states per model	Lead, Invoice, Task, IntakeSession, etc.	5+ incompatible state machines	Replace with constitutional 12-state lifecycle
D-03	Hardcoded foreign keys instead of graph relationships	20+ FK columns across models	Relationships are not queryable, not composable	Replace with `Relationship` kernel records
D-04	No immutable event system	All objects	No audit trail, no reconstruction, no replay	Build immutable Event store
D-05	No versioning	All objects	State cannot be reconstructed	Implement versioning on every mutation
D-06	No unified search	All routes	Search is per-model, per-route	Build unified semantic search engine

Major Deviations (Should Fix Before Genesis)

#	Deviation	Location	Impact	Fix
D-07	No timeline	All objects	Events, observations, communications scattered	Build Timeline kernel
D-08	No observation system	All objects	Objects are black boxes	Implement Observation engine
D-09	Execution not attached to objects	All routes	Business logic lives in route handlers, not on objects	Attach ExecutionPlan to objects
D-10	AI not standardized	CompanionEngine, CoachEngine, CreativeEngine	No standard AIInsight model	Standardize AI output format

Minor Deviations (Can Fix After Genesis)

#	Deviation	Location	Impact	Fix
D-11	No graph-based permissions	Route-level auth checks	Manual permission management	Implement graph permission model
D-12	No workspace composition	Hardcoded layouts	Industry-specific workspaces require custom code	Build composition engine
D-13	No provider lock-in hardening	Runtime config	Currently Flask-only	Abstract runtime interfaces

---

Recommended Refactoring Order

1. **Kernel Record base** — All models inherit from `Record` (D-01)
2. **Relationship graph** — Replace FKs with Relationship records (D-03)
3. **Universal Lifecycle** — 12-state lifecycle on every object (D-02)
4. **Immutable Events** — Event store + event emission (D-04)
5. **Versioning** — Every mutation creates a version (D-05)
6. **Timeline** — Every object owns its timeline (D-07)

7. **Unified Search** — Semantic + full-text search (D-06)
8. **Observation/AI** — Standardized AIInsight (D-08, D-10)
9. **Execution** — Object-attached execution (D-09)
10. **Permissions/Gov** — Graph-based permissions (D-11, D-12)

This report documents the pre-refactoring state. Genesis Reset will implement constitutional compliance.